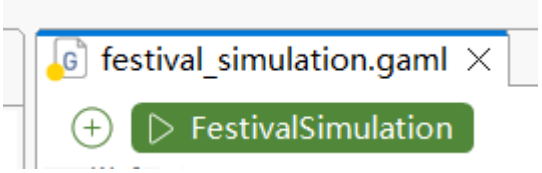
ID2209 Project Report

Group 3
Shunkang Jia, Rui Zhang, Yuning Liu
shunkang, rzha, yuningl @kth.se

Section 1: Running Instruction

Copy the festival_simulation.gaml file into user model section and click the "FestivalSimulation" button.



Section 2: General Overview

In the final project, we have accomplished the basic task as well as the 2 challenges: the BDI Agent and Reinforcement Learning Agent.

We have designed 5 types of guest agent and 2 types of venues. Each guest is rewarded or punished in happiness differently according to the rule defined for its type. In the basic setting, the agents are purely reactive and may randomly choose their behavior. In the BDI agent experiment, they will act based on their beliefs and their intentions. In reinforcement learning experiments, they will figure out their best strategies themselves by learning from previous action results.

Section 3: Festival Base Setting

3.1 Explanation

The basic setting of the festival simulation is explained here.
5 types of guests are defined (categoried by 3 metrics) and 2 type of places are created.

The 3 key metrics are socialability, risk aversion and trust
Sociability (S) defines how much reward/loss agent can benefit from interacting, it is also linked to moving speed and higher
Risk Aversion (RA) defines how much agent is punished from interacting
Trust (T) defines the extent agents may change their belief on how good a stage or bar is based on other agent comments.

Under these 3 metric, we define 5 type agents: Elderly, Child, Student, Worker, Couple.

Agent (Type)	Characteristic	Sociability (S)	Risk Aversion (RA)	Trust (T)
1. Elderly	Hate social, low energy(speed)	0.3 ~ 0.5	0.7 ~ 0.95	0.6~0.85
2. Child	Dynamic, love social	0.7 ~ 1.0	0.05 ~ 0.3	0.7 ~0.95
3. Student	love social	0.6 ~ 0.9	0.2 ~ 0.5	0.5 ~ 0.8
4. Worker	under pressure, do not trust comment	0.4 ~ 0.7	0.5 ~ 0.8	0.3 ~ 0.6
5. Couple	focus on couple, low motivation to social	0.3 ~ 0.6	0.4 ~ 0.7	0.5 ~ 0.8

3.2 Code

3.2.1 Agent Diversity (Types and Traits)

To ensure behavioral variety, we initialize 50 guests with types distributed randomly. The personality traits are not fixed but generated within ranges specific to the guest type.

```
// create 50! guests with distributed types
create Guest number: num_guests {
    location <- {rnd(env_size), rnd(env_size)};

    // Assign type (roughly equal distribution)
    guest_type <- rnd(1, 5);

    // Initialize 3 personality traits based on type
    if (guest_type = 1) { // Elderly
        sociability <- rnd(0.3, 0.5);       // Low-moderate sociability
        risk_aversion <- rnd(0.7, 0.95);    // High risk aversion
        trust <- rnd(0.6, 0.85);            // Relatively trusting
    } else if (guest_type = 2) { // Child
        sociability <- rnd(0.7, 1.0);       // Very social and energetic
        risk_aversion <- rnd(0.05, 0.3);    // Low risk awareness
        trust <- rnd(0.7, 0.95);            // Very trusting
    } else if (guest_type = 3) { // Student
        sociability <- rnd(0.6, 0.9);       // High sociability
        risk_aversion <- rnd(0.2, 0.5);     // Moderate risk tolerance
        trust <- rnd(0.5, 0.8);             // Moderate trust
    } else if (guest_type = 4) { // Office Worker
        sociability <- rnd(0.4, 0.7);       // Moderate sociability
        risk_aversion <- rnd(0.5, 0.8);     // Moderate-high risk aversion
        trust <- rnd(0.3, 0.6);             // Lower trust (skeptical)
    } else { // Type 5: Couple
        sociability <- rnd(0.3, 0.6);       // Lower sociability (focused c
        risk_aversion <- rnd(0.4, 0.7);     // Moderate risk aversion
        trust <- rnd(0.5, 0.8);             // Moderate trust
    }

    // Initialize reputation map for all bars
    loop b over: Bar {
        bar_reputation[b] <- 0.5;
    }
}
```

3.2.2 Interaction Rules

We implemented a complex interaction matrix. When agents are within interaction_distance, they decide to interact based on their sociability. The outcome depends on the compatibility of the two types.

```
} else if (guest_type = 4) { // Office Worker
    interact_prob <- interact_prob - 0.05;  // Tired, less social
    if (other.guest_type = 1) {
        interaction_type <- "neutral";  // Polite but reserved
    } else if (other.guest_type = 2) {
        if (flip(0.4)) { interaction_type <- "positive"; }  // Some like kids
        else { interaction_type <- "negative"; }  // Many are annoyed
    } else if (other.guest_type = 3) {
        if (flip(0.5)) { interaction_type <- "positive"; }  // Remember being
        else { interaction_type <- "neutral"; }
    } else if (other.guest_type = 4) {
        if (flip(0.7)) {
            interaction_type <- "positive";  // Bond over work stress
        } else {
            interaction_type <- "neutral";
        }
    } else if (other.guest_type = 5) {
        if (flip(0.3)) { interaction_type <- "negative"; }  // Jealous of cou
        else { interaction_type <- "neutral"; }
    }
```

3.2.3 Long-Distance FIPA Communication

Agents do not only interact locally. When a guest has a good/bad experience at a Bar, they send a FIPA inform message to their specific peer group

Sending the Message:

```
action share_bar_experience(Bar b, float experience) {
    // Determine message type based on experience
    string rep_status;
    if (experience > 0.5) {
        rep_status <- "good";
    } else {
        rep_status <- "bad";
    }

    // Create message content as a list
    list<string> msg_content <- [b.bar_name, rep_status, string(experience)];

    // Select recipients based on guest type
    list<Guest> recipients <- [];
    int max_recipients <- 5;

    if (guest_type = 3) { max_recipients <- 15; }  // Students share with many fr
    if (guest_type = 4) { max_recipients <- 8; }   // Office workers share modera
    if (guest_type = 1) { max_recipients <- 3; }   // Elderly share with fewer pe
    if (guest_type = 5) { max_recipients <- 2; }   // Couples mostly talk to each

    // Get nearby guests
    list<Guest> nearby <- (Guest where (each != self));
    nearby <- nearby sort_by (each distance_to self);

    int count <- 0;
    loop g over: nearby {
        if (count < max_recipients) {
            recipients <- recipients + g;
            count <- count + 1;
        }
    }

    // Send FIPA INFORM message
    if (length(recipients) > 0) {
        do start_conversation to: recipients performative: "inform" contents: msg
        total_fipa_messages_sent <- total_fipa_messages_sent + 1;
        fipa_cooldown <- 30;
    }
}
```

Receiving and Updating Beliefs: The recipient updates their internal bar_reputation map based on their trust trait.

```
reflex receive_fipa_messages when: use_fipa and (length(informs) > 0) {
    loop msg over: informs {
        list content <- list(msg.contents);

        if (length(content) >= 3) {
            string received_bar_name <- string(content[0]);
            string rep_status <- string(content[1]);
            float sender_experience <- float(content[2]);

            // Find the bar being discussed
            Bar discussed_bar <- nil;
            loop b over: Bar {
                if (b.bar_name = received_bar_name) {
                    discussed_bar <- b;
                }
            }

            if (discussed_bar != nil) {
                // UPDATE INTERNAL BELIEF based on trust trait
                float current_rep <- bar_reputation[discussed_bar];
                float message_rep;
                if (rep_status = "good") {
                    message_rep <- sender_experience + 0.1;
                    if (message_rep > 1.0) { message_rep <- 1.0; }
                } else {
                    message_rep <- sender_experience - 0.1;
                    if (message_rep < 0.0) { message_rep <- 0.0; }
                }

                // Weighted update: trust determines how much to believe
                float weight <- trust * 0.3;
                float new_rep <- current_rep * (1 - weight) + message_rep * weigh
                bar_reputation[discussed_bar] <- new_rep;

                // Type-specific reactions to information
                if (guest_type = 4) { // Office Workers are skeptical
                    bar_reputation[discussed_bar] <- current_rep * 0.8 + new_rep
                } else if (guest_type = 2) { // Children believe everything
                    bar_reputation[discussed_bar] <- current_rep * 0.3 + new_rep
                } else if (guest_type = 1) { // Elderly are moderately trusting
                    bar_reputation[discussed_bar] <- current_rep * 0.6 + new_rep
                }
            }
        }
    }
}
```

3.3 Demonstration

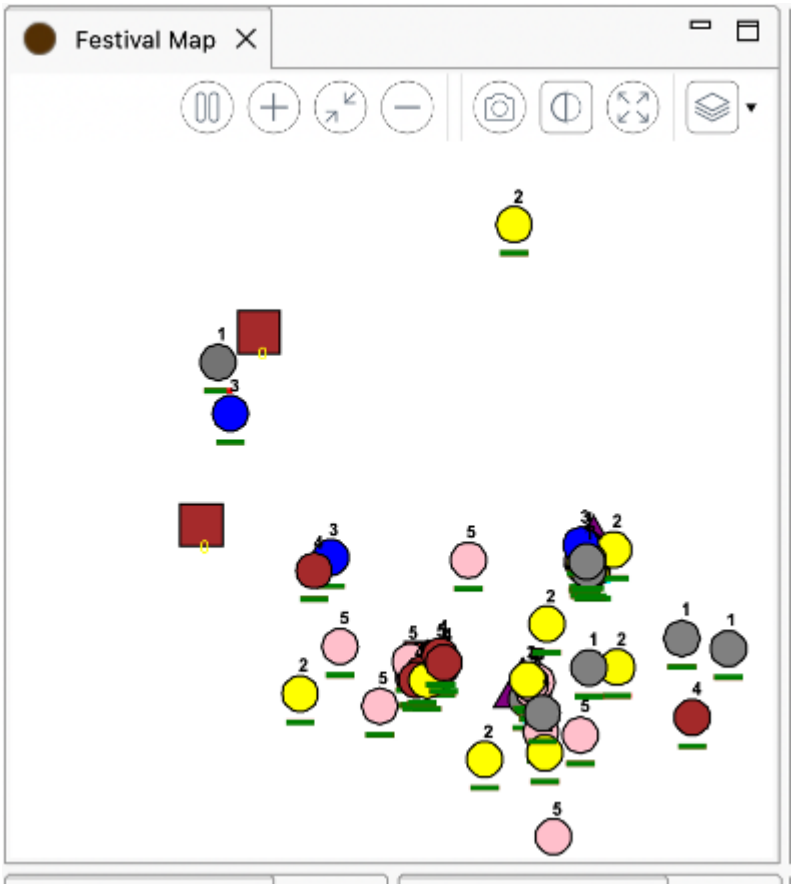
3.3.1 Visualization of Diverse Agents

(1) Input Description

·The simulation is initialized with num_guests = 50.

· The display distinguishes types by color: Grey (Elderly), Yellow (Child), Blue (Student), Brown (Worker), Pink (Couple).

(2) Screenshot



(3) Interpretation

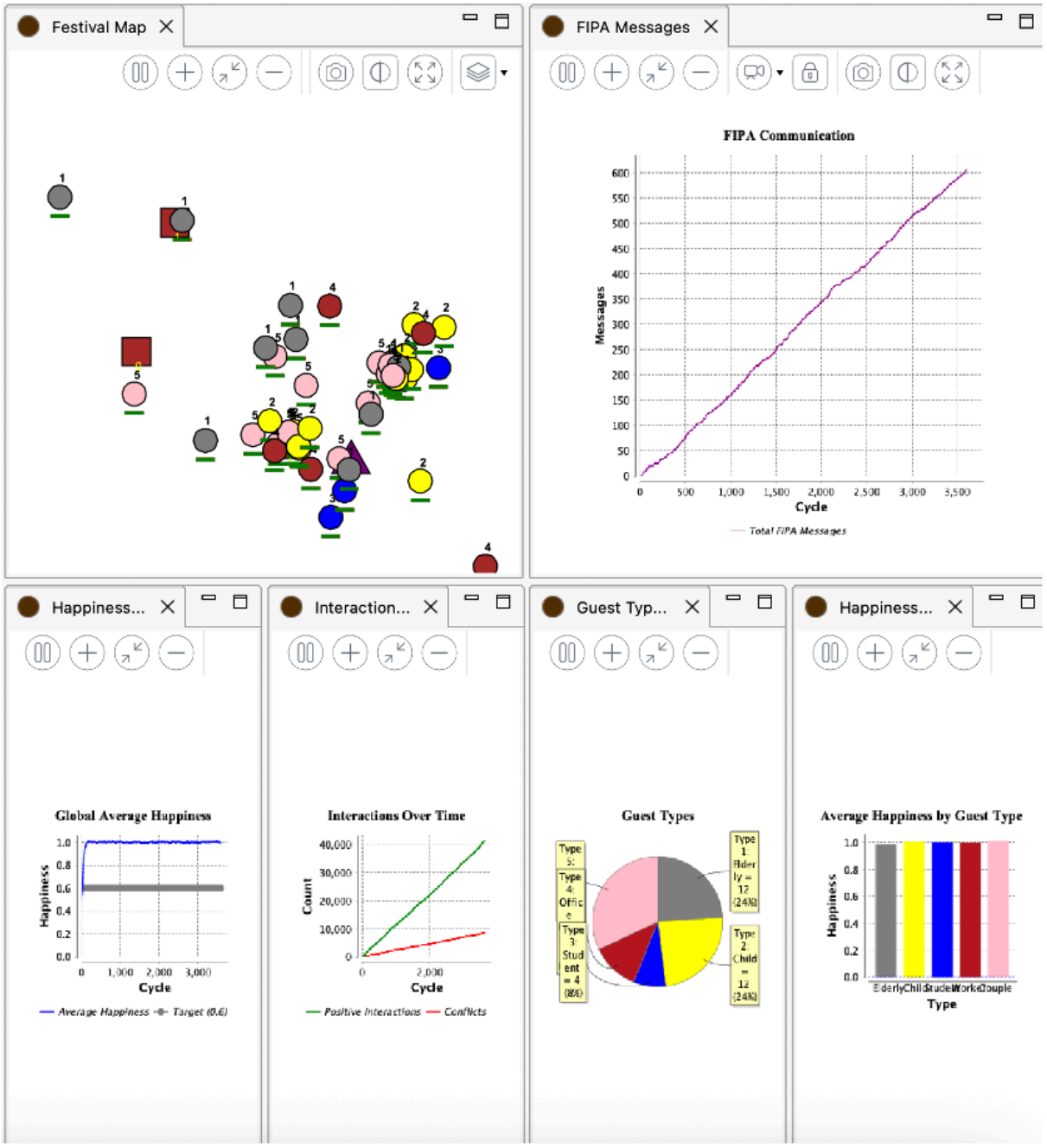
- The screenshot verifies that the environment is populated with 5 distinct agent types. They utilize the wandering and going_to_venue states to navigate between social hubs (Bars) and performance hubs (Stages), confirming the correct initialization of the diverse population.

3.3.2 Global Happiness Analysis (With FIPA)

(1) Input Description

· We run the main FestivalMain experiment for approx. 500 cycles.

(2) Screenshot



(3) Interpretation

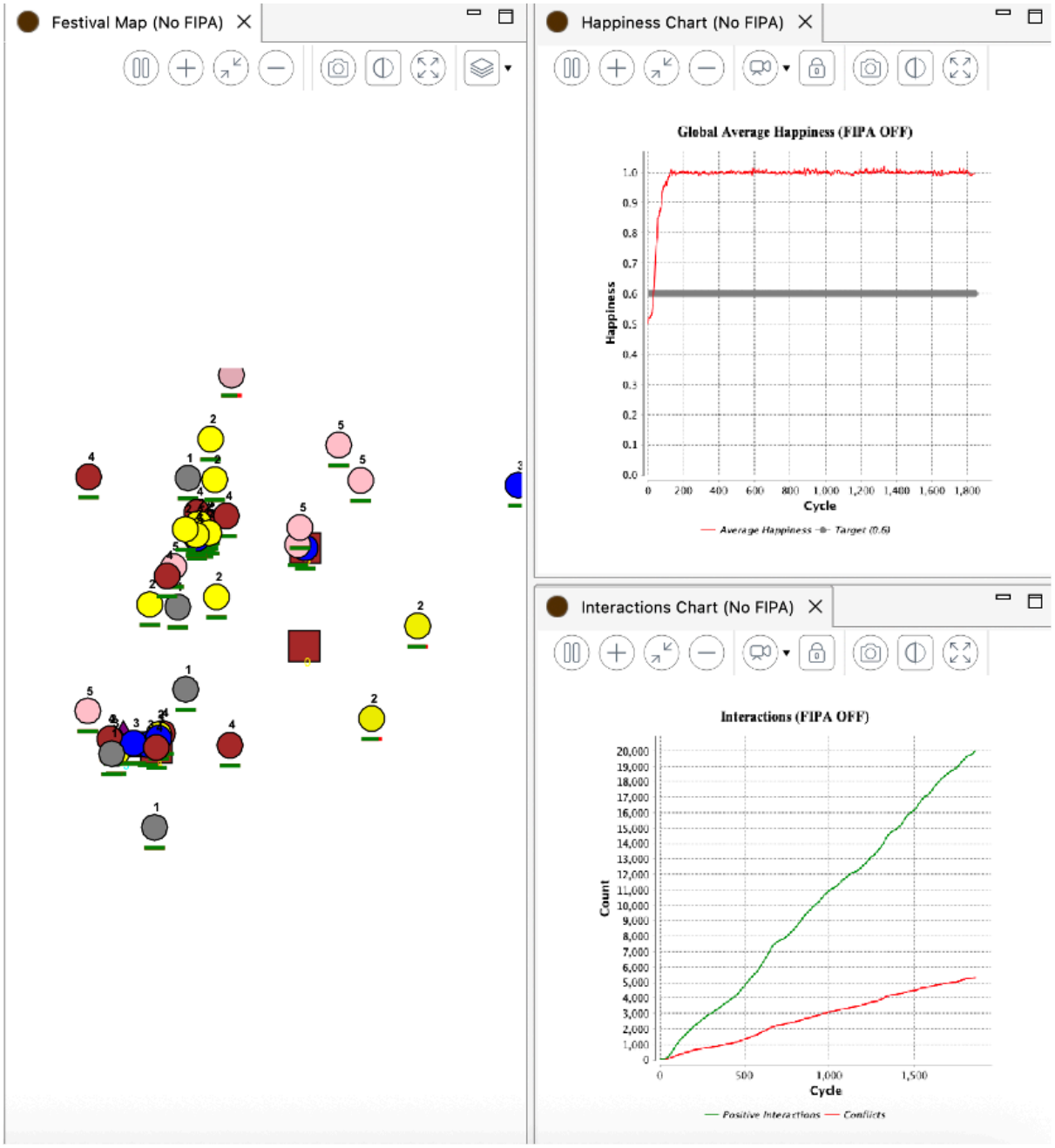
As seen in the "FIPA Communication" chart (top right), the number of messages steadily increases, confirming active information exchange. Consequently, the "Global Average Happiness" (bottom left) rapidly rises and stabilizes at the maximum value of 1.0. This indicates that with FIPA enabled, agents effectively share knowledge to find the best venues and compatible partners, optimizing their experience to perfection.

3.3.3 Comparative Conclusion : Impact of FIPA vs. No FIPA

(1) Experiment Description

To satisfy the requirement of analyzing behavioral impacts, we ran the comparative experiment FestivalNoFIPA to test if social communication changes how happiness evolves over time.

（2）Screenshot



（3）Conclusion：

Adding FIPA behavior changes the system for the better by significantly improving efficiency.

- **Time Analysis:** We compared the growth curves. In the **FIPA-enabled** scenario , the happiness curve spikes vertically, reaching the peak (1.0) in **less than 100 cycles**. In

contrast, the **No-FIPA** scenario exhibits a much slower growth curve, taking approximately **200-300 cycles** to stabilize at the maximum.

- **Verdict:** While both simulations eventually reach high happiness, the FIPA-enabled system is **better**. The "viral" propagation of information allows agents to identify high-quality venues instantly without individual trial-and-error. This proves that distributed communication acts as a catalyst, optimizing the global state 3x faster than non-communicating agents.

Section 4: Challenge 1 Introducing BDI Agent

4.1 Explanation

BDI Architecture Overview

We implemented the Belief-Desire-Intention (BDI) architecture for our festival simulation agents. The BDI model is particularly well-suited for social simulations because it captures how intelligent agents make decisions based on their knowledge of the world (beliefs), their goals (desires), and their commitment to specific actions (intentions).

BDI Components Implementation

1. Beliefs (10 types)

Our agents maintain beliefs about their environment through continuous perception:

- Energy state: `tired`, `energetic`
- Crowd density: `crowded`, `quiet`
- Venue assessment: `good_venue`, `bad_venue`
- Emotional state: `unhappy`, `happy`
- Social context: `others_nearby`, `friend_nearby`

2. Desires (6 types)

Agents have multiple concurrent goals that reflect their guest type and current situation:

- `be_happy` - Universal meta-goal
- `rest` - Elderly, Office Workers
- `socialize` - Children, Students
- `find_fun` - Students, Couples
- `explore` - Children
- `avoid_crowd` - Dynamically generated for risk-averse agents

3. Intentions

Through deliberation, agents select one desire as their current intention based on priority. This commitment guides their actions until the goal is achieved or circumstances change.

4. Actions

Six distinct action implementations execute the selected intention:

- `action_rest` - Recover energy at stages
- `action_socialize` - Interact with nearby guests
- `action_avoid_crowd` - Move away from crowded areas
- `action_find_fun` - Seek high-quality venues
- `action_explore` - Discover new venues
- `action_be_happy` - Meta-action to generate sub-goals

The BDI Deliberation Cycle

Our implementation follows the classical BDI cycle with five distinct steps executed each simulation cycle:

PERCEPTION → UPDATE BELIEFS → APPLY RULES → SELECT INTENTION → EXECUTE ACTION

Step 1: Perception - Agents sense their environment through GAMA's spatial queries and attribute inspection.

Step 2: Update Beliefs - Based on perceptions, agents form beliefs about their current situation. For example, if an agent's energy drops below 0.3, it forms the belief `tired`.

Step 3: Apply BDI Rules - Eight rules generate desires from beliefs and personality traits:

- Rule 1: `tired` → add desire `rest` (highest priority)
- Rule 2: `recovered` → remove desire `rest`
- Rule 3: `energetic` ∧ `high sociability` → add desire `socialize`
- Rule 4: `crowded` ∧ `high risk_aversion` → add desire `avoid_crowd`
- Rule 5: `quiet` ∧ `high sociability` → add desire `socialize`
- Rule 6: `unhappy` → add desire `find_fun`
- Rule 7: `bad_venue` → add desire `explore`
- Rule 8: `friend_nearby` ∧ `social` → add desire `socialize`

Step 4: Select Intention - From all active desires, the agent commits to one intention based on priority ordering (urgent needs like rest take precedence over entertainment).

Step 5: Execute Action - The agent performs the action corresponding to its current intention.

Personality-Driven Behavior

Different guest types exhibit distinct behavioral patterns due to their personality traits:

- **Elderly (Type 1)**: High risk aversion leads to crowd avoidance; preference for rest results in frequent stage visits
- **Children (Type 2)**: High sociability and low risk aversion produce exploratory, social behavior
- **Students (Type 3)**: Balanced approach with strong social desires and venue exploration
- **Workers (Type 4)**: Prioritize rest but conditionally social based on individual sociability
- **Couples (Type 5)**: Focus on entertainment with moderate social interaction

4.2 Code

4.2.1 Core BDI Data Structures

```
species Guest skills: [moving, fipa, simple_bdi] {

    // ======================================================================
    // BASIC ATTRIBUTES
    // ======================================================================
    int guest_type;
    float sociability;
    float risk_aversion;
    float trust;
    float happiness <- 0.5;
    float energy <- 1.0;

    string current_state <- "wandering";
    point target_location;
    Bar current_bar;
    Stage current_stage;
    int time_at_venue <- 0;

    map<Guest, int> last_interaction_cycle;
    map<Guest, float> social_memory <- []; // Track impression of other guests
    int fipa_cooldown <- 0;

    // ======================================================================
    // BDI ARCHITECTURE COMPONENTS
    // ======================================================================

    // BELIEFS: Agent's knowledge about the world
    list<string> my_beliefs <- [];

    // DESIRES: Agent's goals
    list<string> my_desires <- [];

    // INTENTION: Currently committed goal
    string current_intention <- nil;
    string previous_intention <- nil;

    // ======================================================================
    // INITIALIZATION
    // ======================================================================
    init {
        // Initialize type-specific desires
        my_desires <- ["be_happy"];

        if (guest_type = 1) { // Elderly
            my_desires <- my_desires + "rest";
            my_desires <- my_desires + "avoid_crowd";
        } else if (guest_type = 2) { // Child
            my_desires <- my_desires + "socialize";
            my_desires <- my_desires + "explore";
```

4.2.2 BDI Cycle Implementation

Step 1 & 2: Perception and Belief Update

```
// STEP 2: UPDATE BELIEFS - Form beliefs based on perceptions
reflex update_beliefs {
    total_belief_updates <- total_belief_updates + 1;

    // Clear old beliefs (beliefs change based on current perception)
    my_beliefs <- [];

    // BELIEF 1: Energy state
    if (energy < 0.3) {
        my_beliefs <- my_beliefs + "tired";
    } else if (energy > 0.7) {
        my_beliefs <- my_beliefs + "energetic";
    }

    // BELIEF 2: Crowd state
    list<Guest> nearby <- Guest at_distance 15.0;
    if (length(nearby) > 10) {
        my_beliefs <- my_beliefs + "crowded";
    } else if (length(nearby) < 3) {
        my_beliefs <- my_beliefs + "quiet";
    }

    // BELIEF 3: Venue quality (if at a venue)
    if (current_bar != nil) {
        if (current_bar.quality > 0.6) {
            my_beliefs <- my_beliefs + "good_venue";
        } else if (current_bar.quality < 0.4) {
            my_beliefs <- my_beliefs + "bad_venue";
        }
    } else if (current_stage != nil) {
        if (current_stage.performance_quality > 0.7) {
            my_beliefs <- my_beliefs + "good_venue";
        } else if (current_stage.performance_quality < 0.4) {
            my_beliefs <- my_beliefs + "bad_venue";
        }
    }

    // BELIEF 4: Happiness state
    if (happiness < 0.3) {
        my_beliefs <- my_beliefs + "unhappy";
    } else if (happiness > 0.7) {
        my_beliefs <- my_beliefs + "happy";
    }

    // BELIEF 5: Social environment
    list<Guest> very_close <- (Guest at_distance interaction_distance) where (each != self);
    if (length(very_close) > 0) {
        my_beliefs <- my_beliefs + "others_nearby";

        // Check if we know any of them positively
        bool has_friend <- false;
        loop g over: very_close {
            if (social_memory.keys contains g) {
```

Step 3: Rule-Based Desire Generation

```
// STEP 3: GENERATE/UPDATE DESIRES - Based on beliefs and personality
reflex apply_bdi_rules {
    // Rule 1: tired → rest (HIGH PRIORITY)
    if (my_beliefs contains "tired" and not (my_desires contains "rest")) {
        my_desires <- my_desires + "rest";
    }

    // Rule 2: Remove rest desire when recovered
    if (not (my_beliefs contains "tired") and energy > 0.6 and my_desires contains "rest") {
        my_desires <- my_desires - "rest";
    }

    // Rule 3: energetic + social personality → socialize
    if (my_beliefs contains "energetic" and sociability > 0.5 and not (my_desires contains "socialize")) {
        my_desires <- my_desires + "socialize";
    }

    // Rule 4: crowded + risk averse → avoid_crowd
    if (my_beliefs contains "crowded" and risk_aversion > 0.6 and not (my_desires contains "avoid_crowd")) {
        my_desires <- my_desires + "avoid_crowd";
    }

    // Rule 5: quiet + social → socialize
    if (my_beliefs contains "quiet" and sociability > 0.6 and not (my_desires contains "socialize")) {
        my_desires <- my_desires + "socialize";
    }

    // Rule 6: unhappy → find_fun
    if (my_beliefs contains "unhappy" and not (my_desires contains "find_fun")) {
        my_desires <- my_desires + "find_fun";
    }

    // Rule 7: bad_venue → explore (leave and find better)
    if (my_beliefs contains "bad_venue" and not (my_desires contains "explore")) {
        my_desires <- my_desires + "explore";
    }

    // Rule 8: friend_nearby + social → socialize
    if (my_beliefs contains "friend_nearby" and sociability > 0.5 and not (my_desires contains "socialize")) {
        my_desires <- my_desires + "socialize";
    }
}
```

Step 4: Intention Selection

```
// STEP 4: INTENTION SELECTION - Choose which desire to pursue
reflex select_intention {
    string new_intention <- nil;

    // Priority-based selection (most urgent first)
    if (my_desires contains "rest") {
        new_intention <- "rest";
    } else if (my_desires contains "avoid_crowd") {
        new_intention <- "avoid_crowd";
    } else if (my_desires contains "socialize") {
        new_intention <- "socialize";
    } else if (my_desires contains "find_fun") {
        new_intention <- "find_fun";
    } else if (my_desires contains "explore") {
        new_intention <- "explore";
    } else if (my_desires contains "be_happy") {
        new_intention <- "be_happy";
    }

    // Track intention changes
    if (new_intention != current_intention and new_intention != nil) {
        previous_intention <- current_intention;
        current_intention <- new_intention;
        total_intention_changes <- total_intention_changes + 1;
    }
}
```

Step 5: Action Execution

```
// STEP 5: EXECUTE ACTIONS - Based on current intention
reflex execute_intention {
    if (current_intention = "rest") {
        do action_rest;
    } else if (current_intention = "avoid_crowd") {
        do action_avoid_crowd;
    } else if (current_intention = "socialize") {
        do action_socialize;
    } else if (current_intention = "find_fun") {
        do action_find_fun;
    } else if (current_intention = "explore") {
        do action_explore;
    } else if (current_intention = "be_happy") {
        do action_be_happy;
    }
}
```

4.3 Demonstration

4.3.1 Visualization

Our simulation provides clear visual feedback of BDI activity:

Agent Display:

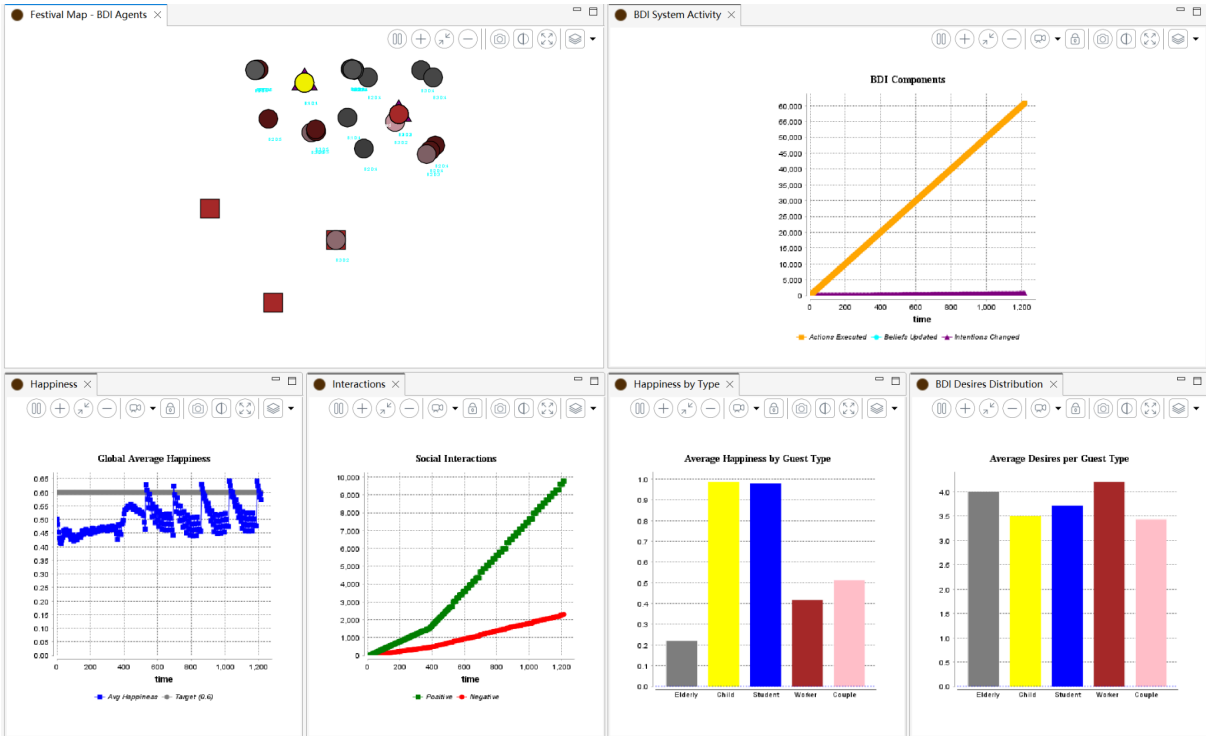
- Agent color intensity represents happiness level
- White text above agent shows **current intention**
- Cyan text shows belief count and desire count (e.g., "B:3 D:4")

Monitoring Charts:

1. **BDI System Activity** - Tracks actions executed, beliefs updated, and intentions changed
2. **BDI Desires Distribution** - Histogram showing average desires per guest type

Console Monitors:

- Real-time display of BDI metrics
- Happiness and interaction statistics



4.3.2 Key Demonstration Points

Continuous Deliberation: BDI Activity chart shows ongoing cognitive processing
Dynamic Adaptation: Intention changes visible in real-time
Personality Influence: Different types exhibit distinct patterns

Goal Persistence: Agents commit to intentions until completion
Emergent Complexity: Simple rules produce realistic social behavior

4.3.3 Conclusion

Our BDI implementation demonstrates that the Belief-Desire-Intention architecture provides a robust framework for modeling intelligent social behavior. The clear separation of beliefs, desires, and intentions enables personality-driven behavior while maintaining computational efficiency. The five-step deliberation cycle produces agents that are both reactive and proactive, mirroring human decision-making in social environments.

Section 5 : Challenge 2 Reinforcement Learning

5.1 Explanation

Reinforcement Learning (RL) is a domain of machine learning where an agent learns to make decisions by performing actions and observing the results. Unlike supervised learning, which relies on labeled datasets, RL is based on interaction with an environment. The agent operates in a loop: it observes the current state, selects an action, receives a numerical "reward" (or penalty) based on the outcome, and updates its strategy to maximize cumulative rewards over time.

In this simulation, the agents use **Q-Learning**, a model-free RL algorithm. The core of Q-Learning is the maintenance of a "Q-Table", which acts as a cheat sheet for the agent. It stores values estimating how good it is to take a specific action. Over time, through a mix of exploration (trying random things) and exploitation (choosing the best-known option), the agent refines these values to converge on an optimal behavioral policy.

All agents are initialized with no prior knowledge of which bar is popular or which stage is good. However, Different types (Elderly, Student, etc.) have different sensitivities to venues (e.g., Students gain more happiness from Bars; Children gain more from Stages) and different decay rates. (reward mechanism is the same as is designed in the base case). Consequently, while they start with the same "blank slate" Q-Matrix, a Student and an Elderly person will learn completely different strategies because the same action yields a different Reward for each of them.

5.2 Code

For simplicity, since we adapt the reward mechanism completely from task1, we only focus on the main reinforcement learning decision and update of the Q table in this part.

5.2.1 Initialization

The agents use Q table to map specific states to action values. We added two distinct Q-tables to the guest agent: `q_venue` for location preferences and `q_interaction` for social strategies. The venue values act as a baseline expectation (0.5), while social

interactions start neutral (0.0). This initialization ensures that agents begin the simulation without pre-existing biases toward specific bars or guest types.

```
// Initialize Q-Tables
// 1. Venue Q-Table: Key = Venue Name, Value = Q-Score
loop b over: Bar { q_venue[b.bar_name] <- 0.5; }
loop s over: Stage { q_venue[s.stage_name] <- 0.5; }

// 2. Interaction Q-Table: Key = "TargetType_Action", Value = Q-Score
// Actions: "Socialize"
// Target Types: "1", "2", "3", "4", "5"
loop i from: 1 to: 5 {
    q_interaction[string(i) + "_Socialize"] <- 0.0;
}
```

5.2.2 Decision Strategy: Epsilon-Greedy

To balance the trade-off between Exploration (discovering new information) and Exploitation (using known information), the model implements an Epsilon-Greedy policy. Controlled by the `epsilon` parameter (0.15), agents have a 15% probability of selecting a random venue or social action, which prevents them from getting stuck in local optima. In the remaining 85% of cases, the agent behaves greedily by iterating through the Q-table and selecting the specific venue or social action associated with the highest historical reward.

```
// Q-LEARNING: VENUE SELECTION
// ==========================================================================
action choose_venue_ql {
    // Epsilon-Greedy
    string chosen_action <- nil;
    list<string> venues <- q_venue.keys;
    bool isexplore <- true;

    if (flip(epsilon)) {
        chosen_action <- venues at rnd(length(venues) - 1); // Explore
    } else {
        // Exploit
        isexplore <- false;
        float max_q <- -9999.0;
        loop v over: venues {
            if (q_venue[v] > max_q) {
                max_q <- q_venue[v];
                chosen_action <- v;
            }
        }
    }

    last_venue_action <- chosen_action;

    // Identify Target
    Bar b_target <- Bar first_with (each.bar_name = chosen_action);
    Stage s_target <- Stage first_with (each.stage_name = chosen_action);

    if (b_target != nil) {
        target_location <- b_target.location;
        current_bar <- b_target;
        current_stage <- nil;
        current_state <- "going_to_venue";
    } else if (s_target != nil) {
        target_location <- s_target.location;
        current_stage <- s_target;
        current_bar <- nil;
        current_state <- "going_to_venue";
    }
```

5.2.3 Reward Calculation and Q-Value Update

The core learning mechanism occurs immediately after an action is completed. The system calculates a `reward` derived from the change in the agent's happiness—for example, a positive value for a high-quality performance or a negative value for a conflict. We then update the Q-value for that specific action using the learning rate `alpha`. This formula incrementally shifts the stored Q-value toward the realized reward, effectively reinforcing successful behaviors and penalizing actions that lead to negative social interactions or poor venue experiences over time.

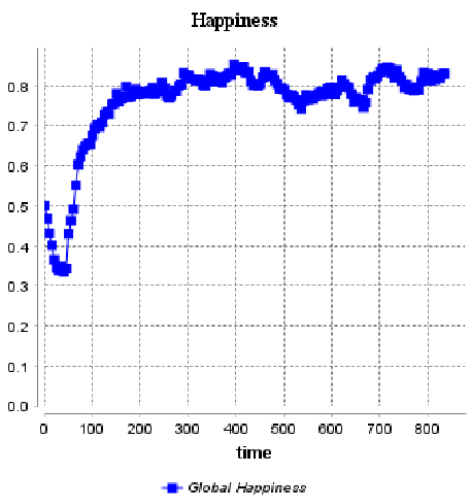
```
// APPLY REWARD
happiness <- happiness + reward;

// UPDATE Q-TABLE (Venue)
// Since leaving is terminal for the venue episode, Q_next is approx 0 (wandering)
float current_q <- q_venue[last_venue_action];
q_venue[last_venue_action] <- current_q + alpha * (reward - current_q);
```

5.3 Demonstration

5.3.1 Average Happiness

The average happiness drops at the very beginning due to the guest are initialized with no prior knowledge of the bars and social activity. However, it soon rises to approximately 0.9 and keeps fluctuating. This is because guests still keep a 15% exploration rate to go randomly even though the agent may have known their best strategy to gain happiness.



5.3.2 Observation: Guest Behavior ID 42

We log the decision process of an agent with id 42, who is an elderly, showing his exploration/exploitation decision as well as his q table.

Initialization:

Initially, the elderly guest has no idea what is good. He flips a coin and decides to exploit the best option and since every bar is identical to him, he chooses the first one Bar 0.

```
============================================
FESTIVAL SIMULATION (Q-LEARNING FULL)
============================================
Decided Venue: Bar_0
Mode: Exploit
Guest 42 (Elderly)
Current Happiness: 0.3899999999999999
Current State: going_to_venue
Target Venue: Bar_0
All Venue Q-Values:
  Venue Bar_0: 0.5
  Venue Bar_1: 0.5
  Venue Bar_2: 0.5
  Venue Stage_0: 0.5
  Venue Stage_1: 0.5
```

Social Behavior

After he arrives at the bar, he starts to socialize: tries 4 times with a type 4 (office worker) and gains little happiness. He suddenly decides to explore other options and social with a type 5 and surprisingly gains much more, leading to his knowledge updated.

```
Socialized with Type 4. Reward: 0.008848501257365103
Guest 42 (Elderly)
Current Happiness: 0.9435107145989947
Current State: at_venue
Top Interaction Q-Values:
  vs Type 1: Soc=0.0
  vs Type 2: Soc=0.0
  vs Type 3: Soc=0.0
  vs Type 4: Soc=8.8E-4
  vs Type 5: Soc=0.0
--------------------------------------------------
Socialized with Type 4. Reward: 0.0019940622110571366
Guest 42 (Elderly)
Current Happiness: 0.9455047768100519
Current State: at_venue
Top Interaction Q-Values:
  vs Type 1: Soc=0.0
  vs Type 2: Soc=0.0
  vs Type 3: Soc=0.0
  vs Type 4: Soc=0.001
  vs Type 5: Soc=0.0
--------------------------------------------------
Socialized with Type 4. Reward: 0.0057066273120200085
Guest 42 (Elderly)
Current Happiness: 0.9808049264318587
Current State: at_venue
Top Interaction Q-Values:
  vs Type 1: Soc=0.0
  vs Type 2: Soc=0.0
  vs Type 3: Soc=0.0
  vs Type 4: Soc=0.00147
  vs Type 5: Soc=0.0
--------------------------------------------------
Socialized with Type 5. Reward: 0.03945903259464504
Guest 42 (Elderly)
Current Happiness: 1.0202639590265037
Current State: at_venue
Top Interaction Q-Values:
  vs Type 1: Soc=0.0
  vs Type 2: Soc=0.0
  vs Type 3: Soc=0.0
  vs Type 4: Soc=0.00147
  vs Type 5: Soc=0.00395
```

Afterwards…

He may go to another bar for entertainment and socializing, but now he has the knowledge of how he feels about bar 0.

```
--------------------------------------------------
Decided Venue: Bar_1
Mode: Exploit
Guest 42 (Elderly)
Current Happiness: 0.995
Current State: going_to_venue
Target Venue: Bar_1
All Venue Q-Values:
  Venue Bar_0: 0.0056
  Venue Bar_1: 0.5
  Venue Bar_2: 0.5
  Venue Stage_0: 0.5
  Venue Stage_1: 0.5
--------------------------------------------------
```

5.3.3 Reinforcement learning result

After many cycles, the guest has his q-table converged to a certain value, it is in line with what we have designed for its social interaction reward mechanism. The elderly (type 1) are more likely to interact with type 1 and type 5 (all interaction is considered positive leading to happiness gain). The q table also shows that type 1 and type 5 have higher q values. This proves the effectiveness of reinforcement learning.

```
Socialized with Type 4. Reward: -0.0012651635574339802
Guest 42 (Elderly)
Current Happiness: 0.9642440269023924
Current State: at_venue
Top Interaction Q-Values:
  vs Type 1: Soc=0.03946
  vs Type 2: Soc=0.02272
  vs Type 3: Soc=0.01395
  vs Type 4: Soc=0.00339
  vs Type 5: Soc=0.03946
```

```
// --- LOGIC TREE START ---
if (guest_type = 1) { // Elderly
    if (other.guest_type = 1) { interaction_type <- "positive"; }
    else if (other.guest_type = 2) { if (flip(0.6)) { interaction_type <- "positive"; } else { interaction_type <- "negative"; } }
    else if (other.guest_type = 3) { if (flip(0.4)) { interaction_type <- "positive"; } else { interaction_type <- "neutral"; } }
    else if (other.guest_type = 4) { interaction_type <- "neutral"; }
    else if (other.guest_type = 5) { interaction_type <- "positive"; }
```

Section 6: Conclusion

In this project, we successfully developed a multi-agent festival simulation using three approaches: In the basic task, we designed a reward mechanism for 5 types of guest agent as well as introducing a reputation mechanism to guide guest agents to choose the best venue, laying the foundation for further bonus challenges. Furthermore, when designing BDI agents, we created agents that balance physiological needs (e.g., resting) and psychological desires (e.g., socializing) based on distinct personality traits. Finally, we equip agents with reinforcement learning ability, specifically Q Learning so that agents can learn optimal venue and interaction preferences during the experiment by themselves with no prior knowledge. The effectiveness is proved by detailed logs of specific agent's decision processes.

There are still some limitations in our experiment which suggest possible improvements in future works. First, currently BDI agents basically rely on hard-coded rules. Possible future work can combine BDI with RL so that agents can both plan long-term goals and adapt to

changes. Second, RL agents are not very flexible because they still somewhat rely on the state machine of{“go to venue”, “wander”, “social”}, and future work can combine all actions in one q table so that agents can achieve higher happiness (exploit the best strategy among all actions).

The difficulties we encountered are mostly how to design the proper reward mechanism. We have struggled with tuning the reward mechanism so that agents are not too "afraid" of negative rewards and stopped exploring. However, the experiment still suggests the negative interactions are not enough and happiness keeps growing to an unreasonable high. Finally we add a threshold for happiness to solve this issue.